

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 08 -

CI/CD

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	15
REFERÊNCIAS.....	16

EMSE

O QUE VEM POR AÍ?

Nessa aula vamos falar sobre como os processos de integração contínua (sigla CI do inglês, continuous integration) e entrega contínua (sigla CD do inglês, continuous delivery/deployment) podem tornar o processo de deploy mais confiável, organizado e consequentemente mais rápido.



HANDS ON

Partindo da branch `boas_praticas_empacotamento_codigo` (https://github.com/rocaabrera/curso_mlops/tree/boas_praticas_empacotamento_codigo) vamos implementar o código mostrado na branch `main` do repositório (https://github.com/rocaabrera/curso_mlops), onde poderemos encontrar o CI (https://github.com/rocaabrera/curso_mlops/blob/main/.github/workflows/CI.yml) e o CD (https://github.com/rocaabrera/curso_mlops/blob/main/.github/workflows/CD.yml) implementados.

SAIBA MAIS

A integração contínua (sigla CI do inglês, continuous integration) e a entrega contínua (sigla CD do inglês, continuous delivery/deployment) são práticas fundamentais no desenvolvimento de software moderno porque permitem um fluxo de trabalho mais eficiente e de alta qualidade.

Na aula anterior, vimos que a qualidade do desenvolvimento local depende diretamente da configuração correta do ambiente pelo desenvolvedor. Entretanto, isso não garante que o código no repositório esteja correto. Com CI vamos resolver esse problema, garantindo que as pessoas desenvolvedoras possam integrar código novo ao repositório principal com frequência, e que pequenas alterações sejam testadas automaticamente. Isso facilitará a detecção precoce de erros, reduzindo o risco de problemas mais complexos no futuro. Além disso, a integração contínua promoverá a colaboração entre equipes, pois o código será constantemente revisado e atualizado, minimizando conflitos e promovendo um ambiente de desenvolvimento mais coeso.

Por sua vez, a entrega contínua (CD) complementa esse processo ao automatizar a implantação de software em ambientes de produção, no nosso caso, a publicação do pacote com o versionamento correto. Isso significa que, sempre que o código passar da etapa de CI, ele poderá ser implantado de forma segura e rápida, sem a necessidade de intervenção manual. Esse nível de automação não apenas acelera o ciclo de desenvolvimento, mas também melhora a qualidade do software, já que permite entregas frequentes e incrementais, garantindo que novas funcionalidades e correções de bugs cheguem aos usuários com maior rapidez e confiabilidade.

Resumindo, as práticas de CI/CD são essenciais para manter a competitividade e a agilidade no desenvolvimento de software, ao mesmo tempo que asseguram a entrega de produtos de alta qualidade.

Existem várias ferramentas para implementar o fluxo de CI/CD, como Jenkins, Git Lab, Azure Pipelines, entre outros. Entretanto, nessa aula vamos utilizar as Actions do Github. Inicialmente vamos criar um diretório **.github** e dentro dele outro diretório chamado **workflows**. Depois, crie um arquivo chamado **test_workflow.yml**. Nesse workflow vamos adicionar o conteúdo mostrado na Figura 1:

```
workflows > ! test_workflow.yml
name: Primeiro Workflow

on:
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - name: Hello World
      run: echo "Criando meu primeiro Workflow"
```

Figura 1 - Exemplo de workflow básico
Fonte: Elaborado pelo autor (2024)

Esse workflow está nos falando para executar o comando **echo “Criando meu primeiro workflow”** em uma máquina Ubuntu quando for realizado um PR (do inglês, **pull request**) para a branch **main** (linha 3-5). Então, basta commitar esse arquivo e fazer um pull request para a branch main, conforme a Figura 2:

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff](#)

base: main ← compare: ci_cd_empacotamento_codigo ✓ Able to merge. These branches can be automatically merged.

Add a title

Ci cd empacotamento codigo

Add a description

Write Preview

Meu primeiro workflow

Markdown is supported Paste, drop, or click to add files

Create pull request

Figura 2 - Abrindo Pull Request da branch com o workflow para main
Fonte: Elaborado pelo autor (2024)

Esse procedimento executará o workflow que “deu match” com o evento especificado ao clicar na aba **Actions** do repositório, conforme mostra a Figura 3. Nesse caso, o pull_request, mas existe uma lista de possíveis eventos no seguinte [link](#).

IMPORTANTE: devido a uma limitação do GitHub, o workflow só será executado se não houver conflitos em relação às branches, conforme mostra a thread no seguinte [link](#).

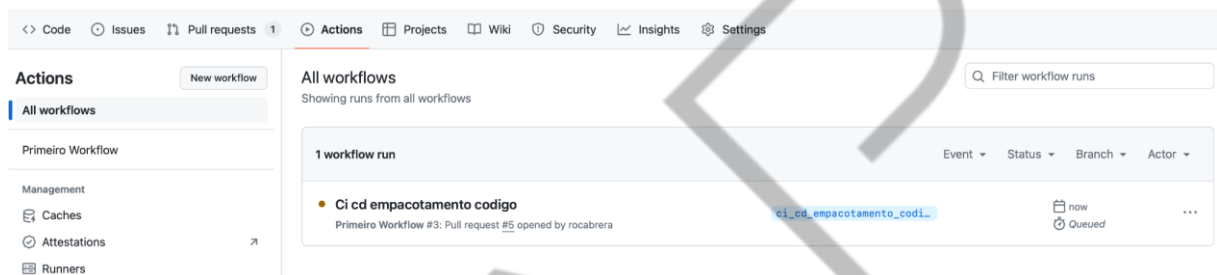


Figura 3 - Action sendo executada
Fonte: Elaborado pelo autor (2024)

Vale notar que o nome dado ao workflow (esquerda da Figura 3) é o especificado no arquivo, enquanto o título do pull request representa o nome da execução do workflow. Ao selecionar o workflow, podemos ver o **job** executado e seus **steps**, conforme mostra a Figura 4:



Figura 4 - Job executado pelo workflow
Fonte: Elaborado pelo autor (2024)

Podemos ver que o que está sendo executado é exatamente o que foi especificado no arquivo. O workflow inicia como se fosse uma máquina nova, então precisamos instalar tudo o que for necessário para executar os comandos. Neste

primeiro workflow, executamos o comando **echo**, em que a máquina com o sistema operacional Ubuntu já possuía. Vamos, agora, aplicar os testes unitários neste workflow, conforme ilustra a Figura 5:

```
! test_workflow.yml M X
.github > workflows > ! test_workflow.yml
1  name: Primeiro Workflow
2
3  on:
4    pull_request:
5      branches: [ "main" ]
6
7  jobs:
8    build:
9      runs-on: ubuntu-latest
10
11     steps:
12       - name: Baixando repositório
13         uses: actions/checkout@v4
14
15       - name: Configurando Python
16         uses: actions/setup-python@v5
17         with:
18           # Semantic version range syntax or exact version of a Python version
19           python-version: '3.11'
20           # Optional - x64 or x86 architecture, defaults to x64
21           architecture: 'x64'
22
23       - name: Instalando o pacote
24         run: pip install -e .
25
26       - name: Instalando dependências
27         run: pip install -r tests/test-requirements.txt
28
29       - name: Executando testes
30         run: pytest tests
```

Figura 5 - Workflow para executar testes unitários no pacote desenvolvido para a aula
Fonte: Elaborado pelo autor (2024)

Perceba que possuímos vários steps:

- Linhas [12-13]: Baixamos o repositório utilizando uma ACTION;
- Linhas [15-21]: Instalamos o Python utilizando outra ACTION;
- Linhas [23-24]: Instalamos o pacote;
- Linhas [26-27]: Instalamos as dependências de teste;
- Linhas [29-30]: Executamos o teste.

Novamente, essas etapas são executadas por uma Action, conforme mostra a Figura 6. Estamos executando um **job** chamado “build”, que executa uma sequência de steps. De forma análoga podemos adicionar a execução do ruff e bandit vistos anteriormente (lembre-se de instalar o requirements correto).

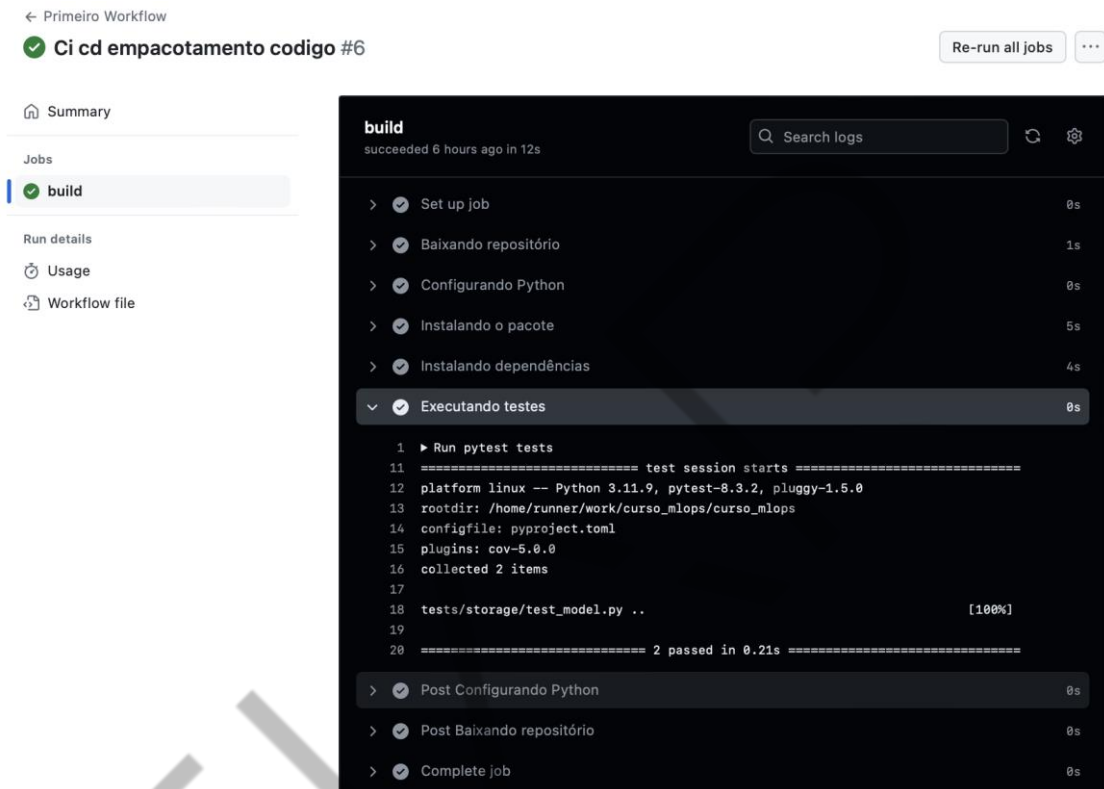


Figura 6 - Action que executa testes unitários no pacote desenvolvido para a aula
Fonte: Elaborado pelo autor (2024)

Caso você queira executar mais de um comando em um mesmo step utilize a sintaxe apresentada na Figura 7:

```
- name: Instalando as dependências
  run: |
    pip install -e .
    pip install -r requirements-dev.txt
```

Figura 7 - Executando vários comandos em um mesmo step
Fonte: Elaborado pelo autor (2024)

Esse processo que foi realizado é o CI da nossa esteira. Portanto, vamos trocar o nome de build para CI e aplicar o resto das boas práticas de desenvolvimento, como por exemplo, a aplicação do método de segurança (bandit) e linter (ruff), conforme mostra a Figura 8:

```

- name: Linter
  run: ruff check

- name: Aplicando pacote de segurança
  run: bandit -c pyproject.toml -r .

```

Figura 8 - Criação de steps de linter e segurança no job de CI
Fonte: Elaborado pelo autor (2024)

Por último, vamos implementar um step no final do job de CI para abrir um PR (do inglês, pull request) da branch atual para a branch de main e mudar o trigger do workflow para criação de uma branch com o prefixo **feature/**. Dessa forma, o workflow de CI é finalizado conforme mostra a Figura 9:

```

1  name: Primeiro Workflow
2
3  on:
4    create:
5      branches:
6        - feature/*
7    push:
8      branches:
9        - feature/*
10
11 jobs:
12   CI:
13     runs-on: ubuntu-latest
14
15     permissions:
16       pull-requests: write
17
18     steps:
19
20       - name: Baixando repositório
21         uses: actions/checkout@v4
22
23       - name: Configurando Python
24         uses: actions/setup-python@v5
25         with:
26           # Semantic version range syntax or exact version of a Python version
27           python-version: '3.11'
28           # Optional - x64 or x86 architecture, defaults to x64
29           architecture: 'x64'
30
31       - name: Instalando as dependências
32         run: |
33           pip install -e .
34           pip install -r requirements-dev.txt
35
36       - name: Executando testes
37         run: pytest tests
38
39       - name: Linter
40         run: ruff check
41
42       - name: Aplicando pacote de segurança
43         run: bandit -c pyproject.toml -r .
44
45       - name: create pull request
46         run: |
47           current_branch=$(git branch --show-current)
48           echo $current_branch
49           gh pr create -B main -H $current_branch --title 'Merging' --body 'Created by Github action'
50         env:
51           GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

```

Figura 9 - Workflow de CI finalizado
Fonte: Elaborado pelo autor (2024)

Vale notar na Figura 9, que é necessário adicionar a permissão de **pull-requests: write** (linha 16), a utilização do CLI do github (gh) para abrir o PR (linhas 45 a 51) e a necessidade de configurarmos o Github para permitir esse tipo de ação na aba de settings do seu repositório, conforme mostra a Figura 10:

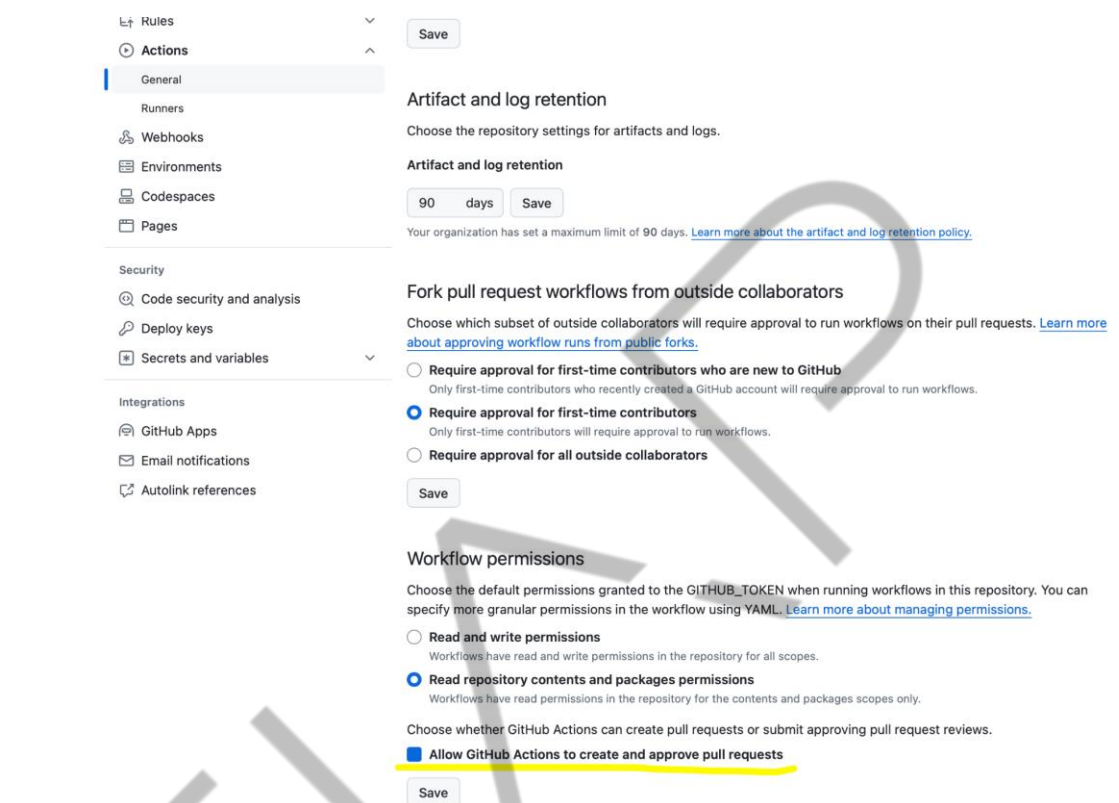


Figura 10 - Habilitando permissão de pull request no github

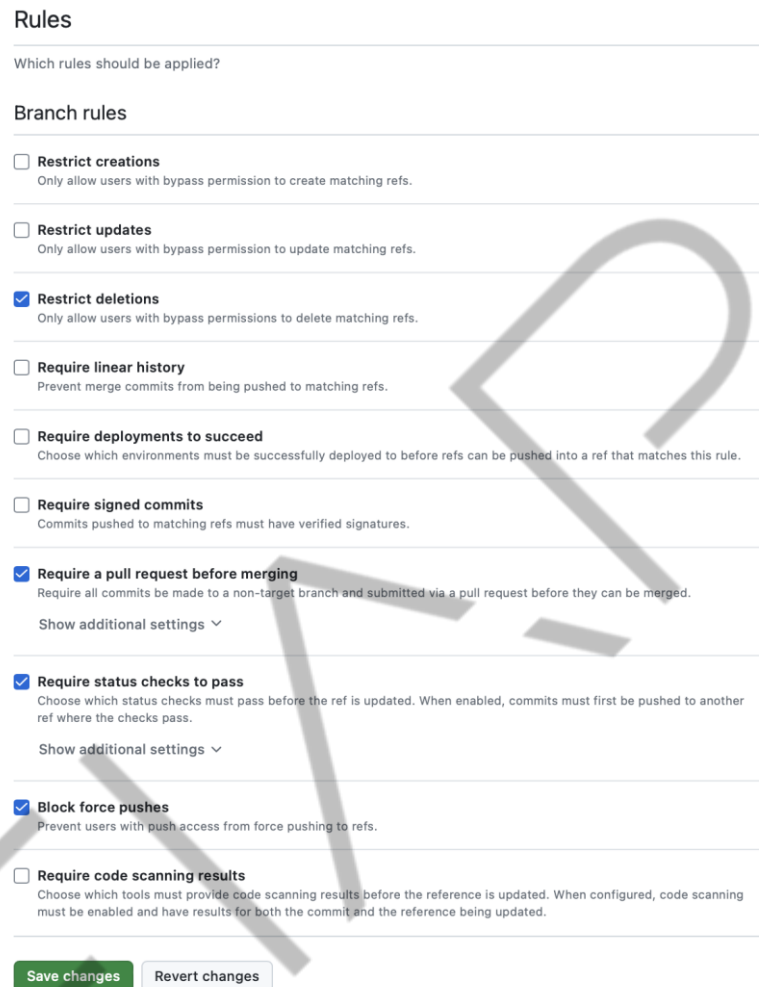
Fonte: Elaborado pelo autor (2024)

Logo o fluxo todo do CI é o seguinte:

- Implementa sua alteração no pacote em uma branch nova com o prefixo feature/;
- Uma action será executada garantindo a execução de testes, linter e segurança.
- Caso não ocorra nenhum erro, será aberto um PR para a branch de main.

Agora, vamos fazer a etapa de CD, isto é, vamos publicar nosso pacote de forma automática toda vez que acontecer uma alteração de código (push na main). Entretanto, precisamos forçar que o CI seja executado antes do CD para garantir que o pacote gerado está funcionando corretamente (passou nos testes), está padronizado (execução do ruff) e tem o nível de segurança esperado (execução do

bandit). Portanto, é interessante no final da implementação lembrar de proteger a branch main, isto é, tornar impossível o push diretamente na branch main configurando um **ruleset** no repositório, conforme mostra a Figura 11:



The screenshot shows the 'Rules' configuration page in GitHub. Under the 'Branch rules' section, several rules are listed with checkboxes. The following rules are checked:

- Restrict deletions**: Only allow users with bypass permissions to delete matching refs.
- Require a pull request before merging**: Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.
- Require status checks to pass**: Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.
- Block force pushes**: Prevent users with push access from force pushing to refs.

At the bottom of the configuration, there are two buttons: 'Save changes' (in green) and 'Revert changes' (in light gray).

Figura 11 - Criando regra (ruleset) para proibir commit diretamente na branch main
Fonte: Elaborado pelo autor (2024)

O workflow de CI utiliza apenas um job. Entretanto, em um mesmo workflow podem existir vários Jobs, que por padrão são executados em paralelo, mas podem ser executados sequencialmente utilizando a palavra-chave **needs**. Visando demonstrar isso, vamos criar um job na esteira de CD chamado `increase_version`, cuja função principal é pegar a última mensagem de commit da branch de feature que abriu o merge e fazer o incremento (**bump**) da tag do git do repositório, conforme mostra a Figura 12:

```

1  name: Primeiro Workflow
2
3  on:
4    push:
5      branches: [ "main" ]
6
7  jobs:
8
9    increase_version:
10     # Som a tag needs o job de CD seria executado de forma paralela.
11     # Dessa forma, a execução do CD depende do CI ser executado com sucesso.
12     runs-on: ubuntu-latest
13     outputs:
14       NEW_VERSION: ${ steps.tag_increase.outputs.NEW_VERSION }
15
16     steps:
17
18       - name: Baixando repositório
19         uses: actions/checkout@v4
20         with:
21           fetch-depth: 0
22           fetch-tags: true
23
24       - name: Incrementa versão
25         id: tag_increase
26         run: |
27           last_commit=$(git log -1 --no-merges --pretty=%B)
28           echo $last_commit
29           if [[ $last_commit == patch* ]]; then
30             NEW_VERSION=$(git describe --tags --abbrev=0 | awk -F. '{OFS="."; $3+=$3; print $0}')
31           elif [[ $last_commit == minor* ]]; then
32             NEW_VERSION=$(git describe --tags --abbrev=0 | awk -F. '{OFS="."; $2+=$2; $3=0; print $0}')
33           elif [[ $last_commit == major* ]]; then
34             NEW_VERSION=$(git describe --tags --abbrev=0 | awk -F. '{OFS="."; sub(/^v/, "", $1); $1+=$1; $2=0; $3=0; print "v"$0}')
35           else
36             echo "Não especifica versao, logo não executa código"
37             exit 1
38           fi
39
40           if [[ $NEW_VERSION == v* ]]; then
41             echo "Precisa começar com v a versão"
42             exit 1
43           fi
44
45           echo "NEW_VERSION=$NEW_VERSION" >> $GITHUB_OUTPUT
46

```

Figura 12 - Job responsável por incrementar a tag do git utilizando mensagem de commit:
Fonte: Elaborado pelo autor (2024)

Além disso, cada job funciona como uma máquina própria e, por isso, eles podem ser executados em paralelo. Portanto, para comunicar essa tag criada ao job seguinte, utilizamos as linhas 13 e 14, pois a linha 45 adiciona a tag à variável de ambiente \$GITHUB_OUTPUT.

O próximo job é denominado **publish**, o qual é responsável por publicar o pacote quando o job anterior terminar, conforme mostra a Figura 13:

```

47  publish:
48    needs: [increase_version]
49
50    runs-on: ubuntu-latest
51
52    environment:
53      name: testpypi
54      url: https://test.pyPI.org/p/ml-communication
55
56    permissions:
57      contents: write
58      id-token: write
59
60    steps:
61
62      - name: Baixando repositório
63        uses: actions/checkout@v4
64        with:
65          fetch-depth: 0
66          fetch-tags: true
67
68      - name: Adiciona Tag
69        run: |
70          git config user.name "$(git log -n 1 --pretty=format:%an)"
71          git config user.email "$(git log -n 1 --pretty=format:%ae)"
72          git tag -a "${ needs.increase_version.outputs.NEW_VERSION }" -m "change version"
73          git tag
74
75      - name: Configurando Python
76        uses: actions/setup-python@v5
77        with:
78          python-version: '3.11'
79          architecture: 'x64'
80
81      - name: Instalando dependências de publicação do pacote
82        run: pip install --U build==1.2.1 twine==5.1.1
83
84      - name: Cria o diretório da distribuição
85        run: python3 -m build
86
87      - name: Publish distribution 📦 to TestPyPI
88        uses: pypa/gh-action-pypi-publish@release/v1
89        with:
90          repository-url: https://test.pyPI.org/legacy/
91
92      - name: Update package tag
93        run: |
94          echo "${ needs.increase_version.outputs.NEW_VERSION }"
95          git push origin "${ needs.increase_version.outputs.NEW_VERSION }"
96

```

Figura 13 - Job responsável por publicar o pacote com a versão nova
Fonte: Elaborado pelo autor (2024)

Para esse job funcionar, é necessário também configurar o TestPyPi para permitir a publicação (forma mais recomendada), conforme mostra a Figura 14:

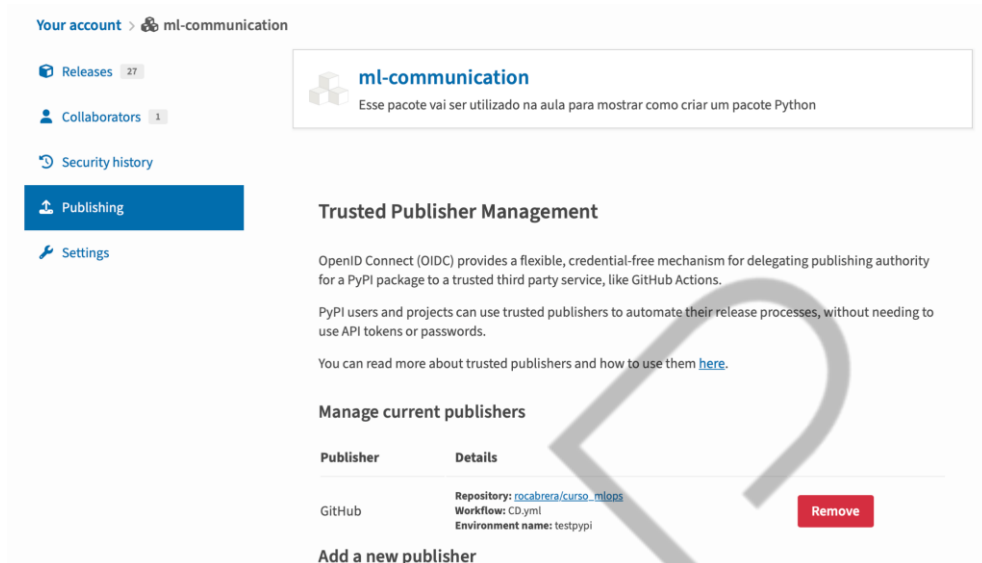


Figura 14 - Configuração do index de publicação para permitir publicação do pacote via workflow
Fonte: Elaborado pelo autor (2024)

O QUE VOCÊ VIU NESTA AULA?

Nessa aula você aprendeu como utilizar o Github Actions para implementar esteiras de CI/CD, entendendo os principais conceitos, aplicações e limitações de workflows, jobs, steps e actions.

EMENDAS

REFERÊNCIAS

GITHUB Docs. **About workflows.** 2024. Disponível em: <<https://docs.github.com/en/actions/writing-workflows/about-workflows#understanding-the-workflow-file>>. Acesso em: 11 set. 2024.

GITHUB Docs. **Creating workflow templates for your organization.** 2024. Disponível em: <<https://docs.github.com/en/actions/sharing-automations/creating-workflow-templates-for-your-organization>>. Acesso em: 11 set. 2024.

GITHUB Docs. **Quick Start.** 2024. Disponível em: <<https://docs.github.com/en/actions/writing-workflows/quickstart>>. Acesso em: 11 set. 2024.

PALAVRAS-CHAVE

Palavras-chave: CI/CD. Entrega Contínua. Automação.

EMSE



POSTECH